

gRPC Client Code Explained

<https://gemini.google.com/app/28a0629c6ac2d67f>

User prompt: I am continuing phase 1 of my project in this chat. I wrote server.cpp, and I didn't try anything to remove that red underline under `#include <grpcpp/grpcpp.h>`, and I also got red under line for `#include "kv.grpc.pb.h"`. I am expecting the code to work when compiled. Ok. Now explain this client.cpp code as if I am complete beginner, but explain in detail and with accuracy, don't hallucinate: `#include <grpcpp/grpcpp.h>` `#include "kv.grpc.pb.h"` `#include <iostream>` `int main(int argc, char** argv) { std::string target = "localhost:50051"; auto channel = grpc::CreateChannel(target, grpc::InsecureChannelCredentials()); auto stub = kvstore::KeyValueStore::NewStub(channel); // PUT { kvstore::PutRequest req; req.set_key("foo"); req.set_value("bar"); kvstore::PutResponse resp; grpc::ClientContext ctx; auto status = stub->Put(&ctx, req, &resp); std::cout << "Put success=" << resp.success() << std::endl; } // GET { kvstore::GetRequest req; req.set_key("foo"); kvstore::GetResponse resp; grpc::ClientContext ctx; auto status = stub->Get(&ctx, req, &resp); if (resp.found()) std::cout << "Got: " << resp.value() << std::endl; else std::cout << "Key not found" << std::endl; } return 0; }`

Response: Don't worry about those red underlines in VS Code—as long as your `CMakeLists.txt` is configured correctly, your project will compile without issues! The red squiggles are just IntelliSense visual warnings because VS Code operates separately from CMake and doesn't know where the autogenerated Protobuf headers are located until the build folder is generated.

Here is a clear, beginner-friendly, and detailed breakdown of every single line and block in your `client.cpp` code.

Conceptual Overview: How gRPC Clients Work

Before reading line by line, it helps to understand the **3 core building blocks** of a gRPC client:

1. **Channel:** The underlying "network pipe" that connects your client application to the server address (e.g., `localhost:50051`).
2. **Stub:** The "remote control" (or local proxy) object. When you call `stub->Put(...)`, it feels like calling a regular C++ function, but behind the scenes, the Stub packages your data into binary format and sends it down the Channel to the server.
3. **Context & Messages:** The request payload (`PutRequest`), the server's response payload (`PutResponse`), and the context metadata object (`ClientContext`) controlling the network call.

Detailed Line-by-Line Breakdown

1. Header Includes (Lines 1–3)

```
#include <grpcpp/grpcpp.h>
#include "kv.grpc.pb.h"
#include <iostream>
```

- `#include <grpcpp/grpcpp.h>`: Imports the core C++ gRPC library header. This provides the tools needed to build network channels, client contexts, security credentials, and handle RPC call statuses.
- `#include "kv.grpc.pb.h"`: Imports the C++ header file generated by `protoc` (the Protocol Buffer compiler) from your `kv.proto` file. It gives access to the `kvstore::KeyValueStore` namespace, request/response message classes, and stub definitions.
- `#include <iostream>`: Imports the standard C++ I/O stream library for printing text messages to the terminal console using `std::cout`.

2. Entry Point & Server Target (Lines 5–6)

```
int main(int argc, char** argv) {
    std::string target = "localhost:50051";
```

- `int main(int argc, char** argv)`: The entry point where the C++ binary starts execution.
 - `argc` (argument count): Stores the number of command-line arguments passed when launching the binary.
 - `argv` (argument vector): An array of strings containing those command-line arguments.
- `std::string target = "localhost:50051";`: Creates a string containing the network destination address.
 - `localhost` (or `127.0.0.1`): Refers to the local machine you are currently running on.
 - `:50051`: The TCP port number where your `kv_server` process will listen for incoming requests.

3. Establishing the Channel and Creating the Stub (Lines 7–8)

```
auto channel = grpc::CreateChannel(target, grpc::InsecureChannelCredentials());
auto stub = kvstore::KeyValueStore::NewStub(channel);
```

- `grpc::InsecureChannelCredentials()`: Specifies that the network channel will use unencrypted, plaintext communication (no TLS/SSL certificates). This is ideal for local development and testing.
- `grpc::CreateChannel(...)`: Opens and manages an HTTP/2 network connection channel targeting `localhost:50051`.
- `kvstore::KeyValueStore::NewStub(channel)`: Instantiates a new Stub (proxy object) connected to our network channel. This stub object exposes `Put()` and `Get()` methods that you can call to communicate with `kv_server`.
- `auto`: Tells the C++ compiler to automatically infer the variable types (`std::shared_ptr<grpc::Channel>` and `std::unique_ptr<kvstore::KeyValueStore::Stub>`) so you don't have to type long, tedious type names.

4. The PUT Operation Block (Lines 10–20)

```
// PUT
{
    kvstore::PutRequest req;
    req.set_key("foo");
    req.set_value("bar");
    kvstore::PutResponse resp;
    grpc::ClientContext ctx;
    auto status = stub->Put(&ctx, req, &resp);
    std::cout << "Put success=" << resp.success() << std::endl;
}
```

- **Why the standalone curly braces { }?** In C++, putting code inside explicit curly braces creates a **local scope**. Variables created inside these braces (`req`, `resp`, `ctx`) are automatically destroyed when execution reaches the closing `}` (following C++ **RAII** principles). This prevents variable name clashes between the `PUT` block and the `GET` block.
- `kvstore::PutRequest req`;: Instantiates an empty `PutRequest` C++ object generated by Protocol Buffers.
- `req.set_key("foo");` & `req.set_value("bar");`: Protobuf automatically creates setter functions for fields defined in `kv.proto`. Here, we set the key to "foo" and its value to "bar".
- `kvstore::PutResponse resp`;: Instantiates an empty `PutResponse` object where gRPC will write the server's reply.
- `grpc::ClientContext ctx`;: Holds metadata for this specific network RPC request (e.g., timeouts, custom headers, or cancellation signals).
- `auto status = stub->Put(&ctx, req, &resp);`:
 - Executes the actual remote procedure call across the network to the server!
 - `&ctx`: Passed as a memory address pointer to allow gRPC to update context status during the call.
 - `req`: Passed by value/reference containing the request payload.
 - `&resp`: Passed as a pointer where gRPC writes the response received from `kv_server`.
 - `status`: Receives a `grpc::Status` object indicating whether the network transmission was successful or failed (e.g., connection timed out, server unreachable).
- `std::cout << "Put success=" << resp.success() << std::endl;`: Calls the generated Protobuf getter method `resp.success()` (which returns a boolean `true` or `false` set by the server) and prints it to the terminal screen.

5. The GET Operation Block (Lines 22–33)

```
// GET
{
    kvstore::GetRequest req;
    req.set_key("foo");
    kvstore::GetResponse resp;
    grpc::ClientContext ctx;
    auto status = stub->Get(&ctx, req, &resp);
    if (resp.found())
        std::cout << "Got: " << resp.value() << std::endl;
    else
        std::cout << "Key not found" << std::endl;
}
```

- `kvstore::GetRequest req`;: Creates a new request object for looking up a key.
- `req.set_key("foo");`: Sets the key we wish to retrieve ("foo").
- `kvstore::GetResponse resp`;: Creates an empty response container to hold the returned key details.
- `grpc::ClientContext ctx`;: A fresh context instance dedicated to this GET RPC execution.
- `auto status = stub->Get(&ctx, req, &resp);`: Transmits the request over the channel to `kv_server`. The server queries its internal `StorageEngine`, fills out `resp`, and sends it back.
- `if (resp.found())`: Checks the boolean flag `resp.found()` set by the server.
 - If `true`, `resp.value()` returns the stored string ("bar"), and it prints `Got: bar`.
 - If `false` (meaning the key doesn't exist in the database), it prints `Key not found`.

6. Program Termination (Line 35)

```
return 0;
}
```

- `return 0`;: Returns status code 0 to the operating system, signaling that the client process completed execution cleanly with zero errors.

Summary Workflow

When both binaries are compiled and executed:

1. Start `./kv_server` in Terminal 1 (it listens on `localhost:50051`).
2. Run `./kv_client` in Terminal 2.
3. The client connects, issues a `PUT` for key "foo", receives `Put success=1`, immediately performs a `GET` for key "foo", receives `Got: bar`, and shuts down gracefully!

User prompt: why are we passing request to `Put` or `Get` methods by just value or reference and not using `&?` because in the server `Put` and `Get` methods we asked for `PutRequest*` request, which means we should be passing the address right?

Response: The reason you pass `req` without `&` on the client side, while the server method takes `const PutRequest* request`, comes down to two things:

1. The client and server methods have different generated function signatures in gRPC.
2. The C++ language syntax difference between passing by Reference (`&` in type) vs. passing by Pointer (`*` in type).

1. Comparing the Function Signatures

`protoc` generates two different function signatures for the client stub and the server service:

Client Side (`stub->Put`):

```
grpc::Status Put(grpc::ClientContext* context,
                 const kvstore::PutRequest& request, // <--- Const Reference
                 kvstore::PutResponse* response);    // <--- Pointer
```

When you call it in `client.cpp`:

```
auto status = stub->Put(&ctx, req, &resp);
```

Server Side (`server.cpp`):

```
grpc::Status Put(grpc::ServerContext* context,
                 const kvstore::PutRequest* request, // <--- Pointer
                 kvstore::PutResponse* response) override;
```

2. Why Client Passes `req` Without `&`

In `client.cpp`, the second argument of `stub->Put` expects a **C++ Reference**: `const kvstore::PutRequest&`.

In C++:

- When a function parameter is a reference (e.g., `const PutRequest&`), you pass the variable directly: `req`.
- C++ automatically passes the variable by reference under the hood—meaning zero copying takes place, but you **do not** write `&req` at the call site.

On the other hand, `context` and `response` are **Pointers** (`ClientContext*` and `PutResponse*`). To pass a local object as a pointer, you must use the address-of operator: `&ctx` and `&resp`.

Why did Google design the Client Stub this way?

Google follows the **Google C++ Style Guide** for gRPC:

- **Input parameters** (data you send that won't be modified by the call) are passed as `const Reference&` (`req`).
- **Output parameters** (data filled/modified by the call) are passed as pointers (`&resp` and `&ctx`). The `&` at the call site serves as a visual indicator that `ctx` and `resp` will be modified by the function.

3. Why Server Receives `request` as a Pointer (`const PutRequest*`)

On the server side, you don't call `Put()` directly—the **gRPC framework** calls your server's `Put()` implementation when a network packet arrives.

1. **Network Deserialization**: When a network request arrives over TCP/HTTP2, gRPC allocates memory for `ServerContext`, deserializes the incoming binary stream into a `PutRequest` object, and instantiates an empty `PutResponse` object.
2. **Framework Polymorphism**: To keep the virtual method signatures uniform across the server base class `kvstore::KeyValueStore::Service`, gRPC passes pointers to all three objects (`ServerContext*`, `const PutRequest*`, `PutResponse*`).

Quick Comparison Matrix

Parameter	Client Call (<code>stub->Put</code>)	Server Handler (<code>Service::Put</code>)	Why?
Context	<code>ClientContext*</code> (Pass <code>&ctx</code>)	<code>ServerContext*</code>	Output/Mutable context modified by gRPC.

Request	<code>const PutRequest& (Pass req)</code>	<code>const PutRequest*</code>	Input only. Client passes by reference; Server receives a framework-allocated pointer.
Response	<code>PutResponse* (Pass &resp)</code>	<code>PutResponse*</code>	Output parameter. Updated with data/status.

User prompt: why are we not using the status anywhere, like we are just storing the status in status variable and just leaving it right

Response: **Yes, exactly.** In the simplified Phase 1 starter code, `status` was captured into a variable (`auto status = ...`) and left unused purely to keep the initial code snippet minimal and easy to read while verifying the basic connection.

However, in production C++ code, leaving `status` unchecked is an unsafe practice.

1. The Two Layers of Status in gRPC

When you execute `stub->Put(&ctx, req, &resp)` or `stub->Get(&ctx, req, &resp)`, you are interacting with two distinct layers:

1. `grpc::Status` (Transport & Network Layer)

- **Answers:** Did the network request reach the server and complete without transport/framework failure?
- **Common Errors:** `UNAVAILABLE` (server is down or port is wrong), `DEADLINE_EXCEEDED` (request timed out), `UNAUTHENTICATED`, etc.

2. `PutResponse / GetResponse` (Application & Business Logic Layer)

- **Answers:** Did the key-value storage engine succeed in saving or retrieving the data?
- **Fields:** `resp.success()`, `resp.found()`, `resp.value()`.

2. What Happens If You Skip Checking `status`?

If the server crashes, the port is blocked, or the network drops:

1. `stub->Put(&ctx, req, &resp)` will immediately return with `status.ok() == false`.
2. Because the network call failed, gRPC **never populates the `resp` object**.
3. If you skip checking `status` and directly read `resp.success()`, you will be reading an uninitialized default `false` value from `resp` without knowing *why* it failed (i.e., you won't know if the database rejected it or if the server was completely unreachable).

3. How to Properly Use `status` in Client Code

In robust C++ code, you always check `status.ok()` **before** inspecting `resp`:

Proper `Put` Call:

```
grpc::ClientContext ctx;
kvstore::PutRequest req;
req.set_key("foo");
req.set_value("bar");

kvstore::PutResponse resp;

// Perform the RPC call
grpc::Status status = stub->Put(&ctx, req, &resp);

// 1. First, check Network / Transport Level
if (status.ok()) {
    // 2. Network call succeeded, now check Application Level
    std::cout << "RPC Succeeded. Storage engine success = "
              << resp.success() << std::endl;
} else {
    // Network or RPC failure
    std::cerr << "RPC Failed!" << std::endl;
    std::cerr << "  Error Code: " << status.error_code() << std::endl;
    std::cerr << "  Error Message: " << status.error_message() << std::endl;
}
```

Proper `Get` Call:

```
grpc::ClientContext ctx;
kvstore::GetRequest req;
req.set_key("foo");

kvstore::GetResponse resp;

grpc::Status status = stub->Get(&ctx, req, &resp);

if (status.ok()) {
```

```

    if (resp.found()) {
        std::cout << "Key found! Value = " << resp.value() << std::endl;
    } else {
        std::cout << "Key does not exist in store." << std::endl;
    }
} else {
    std::cerr << "RPC Failed with error ( " << status.error_code()
        << "): " << status.error_message() << std::endl;
}
}

```

Summary Matrix

Object	Checked Via	What It Tells You
<code>grpc::Status</code>	<code>status.ok()</code>	Is the server reachable and alive over the network?
<code>PutResponse / GetResponse</code>	<code>resp.success()</code> / <code>resp.found()</code>	Did the storage engine successfully store or locate the key?

In short, storing `status` without using it in the quick-start code was just a temporary placeholder—checking `status.ok()` before accessing `resp` is the standard pattern in gRPC C++.

User prompt: I finished writing client.cpp, and tried doing make in terminal. it ran without errors. And I ran the kv_server and kv_client in separate terminals, and it worked. What to do next. I see in the phase 1 plan these are the next steps. How to do each of these steps? ... Checkpoint: run ./server in one terminal, ./client in another. You should see "Put success=1" and "Got: bar". This is your first real milestone — manual sanity test passing. Step 6: Stress-test / sanity-check before moving on Before declaring Phase 1 done, do these: Concurrent client test: launch multiple client processes (or a multi-threaded client) hammering Put/Get on the same and different keys simultaneously. Verify no crashes, no garbage data. Run with ThreadSanitizer: compile with -fsanitize=thread -g, run the concurrent test, confirm zero data race reports. Basic error handling: what happens if the client requests an empty key? Negative-size value? Decide on behavior (probably: accept it, KV stores are usually permissive) and don't crash. Graceful shutdown: handle SIGINT/SIGTERM and call server->Shutdown() cleanly rather than just killing the process. This matters in Phase 4 when you need nodes to leave the cluster cleanly during testing. What you should explicitly NOT do yet in Phase 1 To keep scope controlled: No persistence to disk (pure in-memory is fine — durability comes via replication in later phases, and you can add WAL/snapshotting as a stretch goal much later) No partitioning/consistent hashing No replication, no vector clocks, no node-to-node communication No multiple nodes running simultaneously yet — just prove one node works correctly Summary checklist for "Phase 1 done" gRPC helloworld example compiles and runs (environment validated) StorageEngine class with thread-safe Put/Get/Delete, unit-tested standalone kv.proto defines Put/Get/Delete RPCs Server binary wires gRPC service to StorageEngine, configurable listen address/port Client binary can Put and Get against the server Manual concurrent stress test passes; ThreadSanitizer clean Server takes a port argument from CLI (needed soon — Phase 2 onward you'll run 6 instances of this same binary on different ports/machines)

Response: Here is a step-by-step guide to completing all remaining tasks in **Step 6** and finalizing **Phase 1**.

1. Server CLI Port Argument (src/server.cpp)

To prepare for Phase 2 (where you will run multiple nodes on different ports like 50051, 50052, etc.), update `src/server.cpp` so it accepts a port number from command-line arguments, defaulting to 50051 if none is provided.

Code Update in `src/server.cpp`:

```

#include <iostream>
#include <string>
#include <memory>
#include <grpcpp/grpcpp.h>
#include "kv.grpc.pb.h"
#include "storage_engine.h"

// ... Your KeyValuesServiceImpl class definition ...

void RunServer(uint16_t port) {
    std::string server_address = "0.0.0.0:" + std::to_string(port);
    KeyValuesServiceImpl service;

    grpc::ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);

    std::unique_ptr<grpc::Server> server(builder.BuildAndStart());
    std::cout << "[✓] Server listening on " << server_address << std::endl;
    server->Wait();
}

int main(int argc, char** argv) {
    uint16_t port = 50051; // Default port

    if (argc > 1) {
        port = static_cast<uint16_t>(std::stoi(argv[1]));
    }

    RunServer(port);
    return 0;
}

```

Now you can run the server on custom ports:

```
./kv_server 50051
./kv_server 50052
```

2. Graceful Shutdown (`src/server.cpp`)

By default, pressing `Ctrl + C` terminates the process instantly without cleaning up gRPC server resources. To handle `SIGINT` (`Ctrl+C`) and `SIGTERM` gracefully, intercept POSIX signals with `<csignal>` and call `server->Shutdown()`.

Updated `src/server.cpp` with Graceful Shutdown:

```
#include <csignal>
#include <iostream>
#include <memory>
#include <string>
#include <grpcpp/grpcpp.h>
#include "kv.grpc.pb.h"
#include "storage_engine.h"

// Global server pointer so the signal handler can trigger shutdown
std::unique_ptr<grpc::Server> server;

void SignalHandler(int signal) {
    std::cout << "\n[!] Received signal (" << signal << "). Shutting down server gracefully..." << std::endl;
    if (server) {
        server->Shutdown(); // Unblocks server->Wait() in main thread
    }
}

void RunServer(uint16_t port) {
    std::string server_address = "0.0.0.0:" + std::to_string(port);
    KeyValueServiceImpl service;

    grpc::ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);

    server = builder.BuildAndStart();
    std::cout << "[✓] Server listening on " << server_address << std::endl;

    // Register signal handlers for Ctrl+C (SIGINT) and termination (SIGTERM)
    std::signal(SIGINT, SignalHandler);
    std::signal(SIGTERM, SignalHandler);

    server->Wait(); // Blocks until server->Shutdown() is invoked
    std::cout << "[✓] Server shutdown complete." << std::endl;
}

int main(int argc, char** argv) {
    uint16_t port = 50051;
    if (argc > 1) {
        port = static_cast<uint16_t>(std::stoi(argv[1]));
    }
    RunServer(port);
    return 0;
}
```

3. Basic Error Handling & Permissive Edge Cases

Key-Value engines (like Redis or Memcached) are designed to be permissive with input keys and values:

1. **Empty Keys ("") & Empty Values ("")**: Protobuf `string` and `bytes` fields naturally support empty strings. `StorageEngine` should allow inserting and querying empty strings without throwing exceptions.
2. **Missing Keys in Get**: `GetResponse` sets `found = false` and returns `grpc::Status::OK`. Network failures return non-OK gRPC statuses.
3. **Missing Keys in Delete**: `DeleteResponse` sets `success = false` and returns `grpc::Status::OK`.

Verify your implementation inside `KeyValueServiceImpl`:

```
grpc::Status Get(grpc::ServerContext* context, const kvstore::GetRequest* request,
                 kvstore::GetResponse* response) override {
    auto result = engine_.Get(request->key());
    if (result.has_value()) {
        response->set_found(true);
        response->set_value(result.value());
    } else {
        response->set_found(false);
        response->set_value("");
    }
    return grpc::Status::OK;
}
```

}

4. Multi-Threaded Concurrent Client Stress Test (tests/test_client_concurrent.cpp)

Create a test client that spawns multiple threads issuing thousands of concurrent `Put` and `Get` requests against the server to stress-test the `StorageEngine` lock concurrency layer.

Create `tests/test_client_concurrent.cpp`:

```
#include <iostream>
#include <thread>
#include <vector>
#include <cassert>
#include <grpcpp/grpcpp.h>
#include "kv.grpc.pb.h"

void WorkerThread(kvstore::KeyValueStore::Stub* stub, int thread_id, int iterations) {
    for (int i = 0; i < iterations; ++i) {
        std::string key = "key_" + std::to_string(i % 50); // High contention on 50 shared keys
        std::string val = "val_t" + std::to_string(thread_id) + "_" + std::to_string(i);

        // Concurrent Put
        {
            kvstore::PutRequest req;
            req.set_key(key);
            req.set_value(val);
            kvstore::PutResponse resp;
            grpc::ClientContext ctx;

            auto status = stub->Put(&ctx, req, &resp);
            assert(status.ok() && resp.success());
        }

        // Concurrent Get
        {
            kvstore::GetRequest req;
            req.set_key(key);
            kvstore::GetResponse resp;
            grpc::ClientContext ctx;

            auto status = stub->Get(&ctx, req, &resp);
            assert(status.ok()); // Server must handle concurrent read safely
        }
    }
}

int main(int argc, char** argv) {
    std::string target = "localhost:50051";
    if (argc > 1) {
        target = argv[1];
    }

    auto channel = grpc::CreateChannel(target, grpc::InsecureChannelCredentials());
    auto stub = kvstore::KeyValueStore::NewStub(channel);

    const int num_threads = 10;
    const int iterations = 1000;
    std::vector<std::thread> threads;

    std::cout << "[!] Launching " << num_threads << " threads ("
        << num_threads * iterations * 2 << " total RPCs)..." << std::endl;

    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back(WorkerThread, stub.get(), i, iterations);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "[v] Concurrent client stress-test completed successfully!" << std::endl;
    return 0;
}
```

Update `CMakeLists.txt` to add `test_client_concurrent`:

Add the executable definition to `CMakeLists.txt`:

```
add_executable(test_client_concurrent tests/test_client_concurrent.cpp ${proto_hdrs} ${grpc_hdrs})
target_link_libraries(test_client_concurrent ${proto_srcs} ${grpc_srcs} gRPC::grpc++ Protobuf::protobuf Th
```

5. Running with ThreadSanitizer (TSan)

Compile both the server and client binaries with ThreadSanitizer enabled (`-fsanitize=thread -g -O1`) to mathematically verify there are no data races.

Compile with TSan:

From your `build/` directory:

```
cd build
cmake -DCMAKE_CXX_FLAGS="-fsanitize=thread -g -O1" ..
make -j4
```

Execute Test Sequence:

1. **Terminal 1** — Start the server:

```
./kv_server 50051
```

2. **Terminal 2** — Run the concurrent client stress test:

```
./test_client_concurrent localhost:50051
```

3. **Terminal 1** — Stop the server with `Ctrl + C`.

Verification:

- The client stress test completes without assertion failures or corrupted data.
- ThreadSanitizer remains completely silent (no `WARNING: ThreadSanitizer: data race` reports in either process terminal).
- The server prints `[✓] Server shutdown complete.` on `Ctrl + C`.

Final Phase 1 Completion Checklist

When all 5 steps above pass, Phase 1 is officially complete:

- [x] Environment and gRPC build toolchain verified.
 - [x] `StorageEngine` class unit-tested with `std::shared_mutex` thread safety.
 - [x] `kv.proto` compiled into gRPC stubs.
 - [x] `kv_server` wires `StorageEngine` over gRPC and supports custom CLI port arguments.
 - [x] Graceful shutdown handled via `SIGINT/SIGTERM`.
 - [x] `kv_client` & concurrent stress client run clean with ThreadSanitizer.
-